

Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting¹

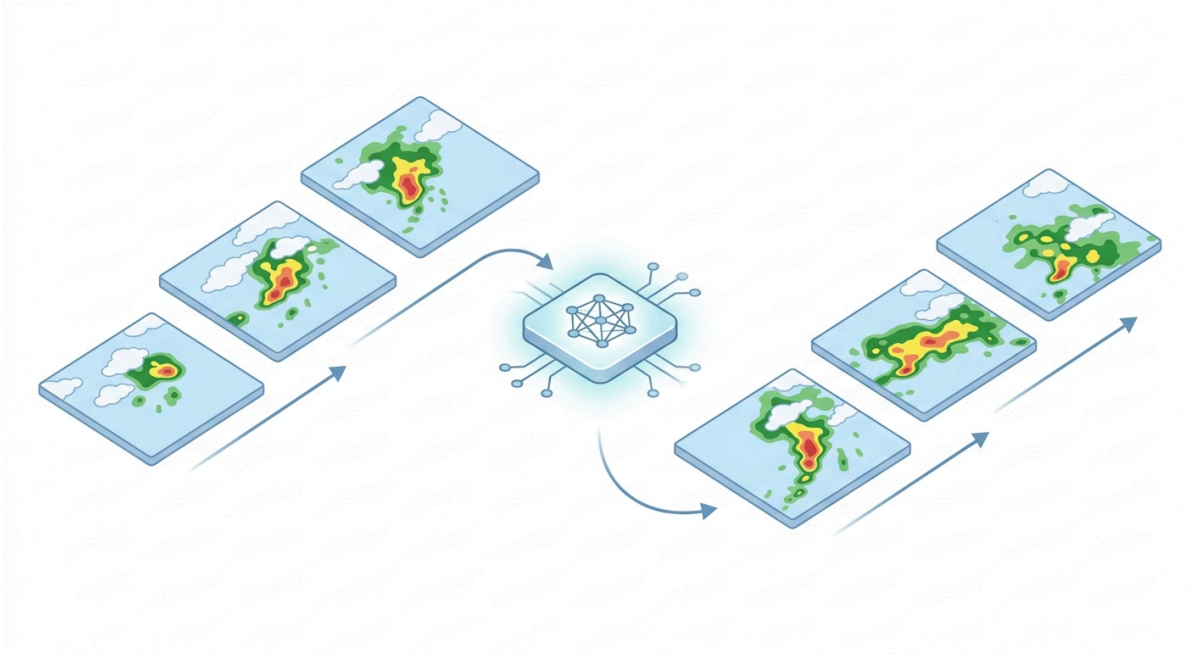
阅读笔记

文献信息

- **论文标题：**Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting
- **作者：**Xingjian Shi, Zhourong Chen, Hao Wang, Dit-Yan Yeung, Wai-kin Wong, Wang-chun Woo
- **DOI/开源链接：**[arXiv:1506.04214](https://arxiv.org/abs/1506.04214)

核心问题与动机

核心科学问题：预测未来几小时的降水分布（时空序列预测），利用雷达回声波序列，高精度地预测局部地区未来短时间（大概两个小时以内）内的降雨强度。



图一²：研究背景与总体引用场景

现有技术的不足：

- **传统气象方法的局限性：**传统的光流法，一种经典的计算机视觉方法，其核心假设为图像中的同一个目标特征在连续两帧之间保持着亮度恒定、形状不变且只发生平移。但是在真实的气象场景中，云层并不是刚体，时刻发生变化，导致亮度和形状时刻快速变化。这些变换高度非线性，导致光流法根本没办法应对复杂的动态过程。

- **传统的FC-LSTM的局限性：** FC-LSTM为带全连接层的LSTM，主要擅长于处理一维时序数据。当把二维的雷达图像数据输入时候必须转换为一维向量，导致图像中的空间拓扑结构（比如哪里是高压中心、哪里是气团锋面）消失，模型无法知道像素之间的关系。并且将二维图像转换为一维向量输入后，全连接层会发生参数爆炸，导致参数冗余、训练效率低、模型过拟合泛化能力差等问题。

对动态变化的二维雷达回波图进行数学建模

在降水临近预报中，在每个时刻获得二维雷达回波图。

- 二维雷达回波图看做 $M \times N$ 的空间网格
- 雷达图上的每个像素作为cell
- 将雷达图进行裁切，得到每一个小块Patch，每个Patch内的cell具有不同的特征数
- 特征数 P ，代表该cell在同一时刻的多个测量值，包括风速、温度等气象数据

之后在任意时刻 t ，都可以用一个三维张量 $\mathcal{X} \in \mathbb{R}^{P \times M \times N}$ 表示：

- 第一维： P 个特征值/测量值
- 第二维： M 行空间网格
- 第三维度： N 列空间网格

在一段时间后得到张量序列。

之后，用过去 J 个时刻（包含当下）的张量序列，来对未来 K 个时刻的序列进行空间序列预测。

$$\tilde{\mathcal{X}}_{t+1}, \dots, \tilde{\mathcal{X}}_{t+K} = \arg \max_{\mathcal{X}_{t+1}, \dots, \mathcal{X}_{t+K}} p(\mathcal{X}_{t+1}, \dots, \mathcal{X}_{t+K} \mid \hat{\mathcal{X}}_{t-J+1}, \hat{\mathcal{X}}_{t-J+2}, \dots, \hat{\mathcal{X}}_t)$$

- $\tilde{\mathcal{X}}_{t+1}, \dots, \tilde{\mathcal{X}}_{t+K}$ ：对未来 K 个时刻的预测目标
- $\arg \max$ ：所有可能的未来序列中，条件概率最大的那一个
- $p(\dots \mid \dots)$ ：已知过去 J 个观测 $\hat{\mathcal{X}}_{t-J+1}, \hat{\mathcal{X}}_{t-J+2}, \dots, \hat{\mathcal{X}}_t$ 的前提下，未来 K 个序列出现的概率

最终，达到用过去 J 帧雷达图来预测未来 K 帧雷达图的时空序列预测目的，将预测问题转化为最大化条件概率的最优序列求解问题。

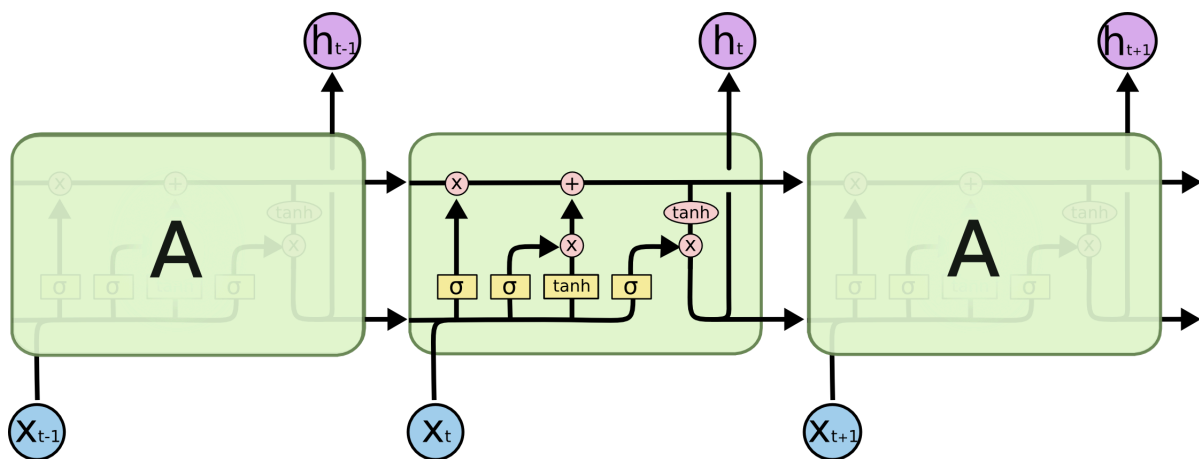
从FC-LSTM到ConvLSTM

FC-LSTM，最基础的LSTM模型，从输入到输出都为一维向量，适用于捕捉时间相关性即时序依赖。

如果在预测未来降水分布中采取FC-LSTM，在处理2D雷达图时，会把2D特征展开为1D向量，丢失空间结构信息的同时，全连接会把所有空间位置的特征全连接而忽略掉局部性带来大量冗余的计算。

于是，该论文把FC-LSTM中的**所有全连接操作，即矩阵逐元素相乘，替换为卷积操作**。这也是该论文的核心创新点。

FC-LSTM只能对**时序依赖**进行建模，但是ConvLSTM能同时对**时序依赖和空间依赖**进行建模



图二³：FC-LSTM核心结构

在FC-LSTM中，核心的公式为：

- $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci} \circ c_{t-1} + b_i)$
- $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf} \circ c_{t-1} + b_f)$
- $\tilde{c}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$
- $c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$
- $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co} \circ c_t + b_o)$
- $h_t = o_t \circ \tanh(c_t)$

其中：

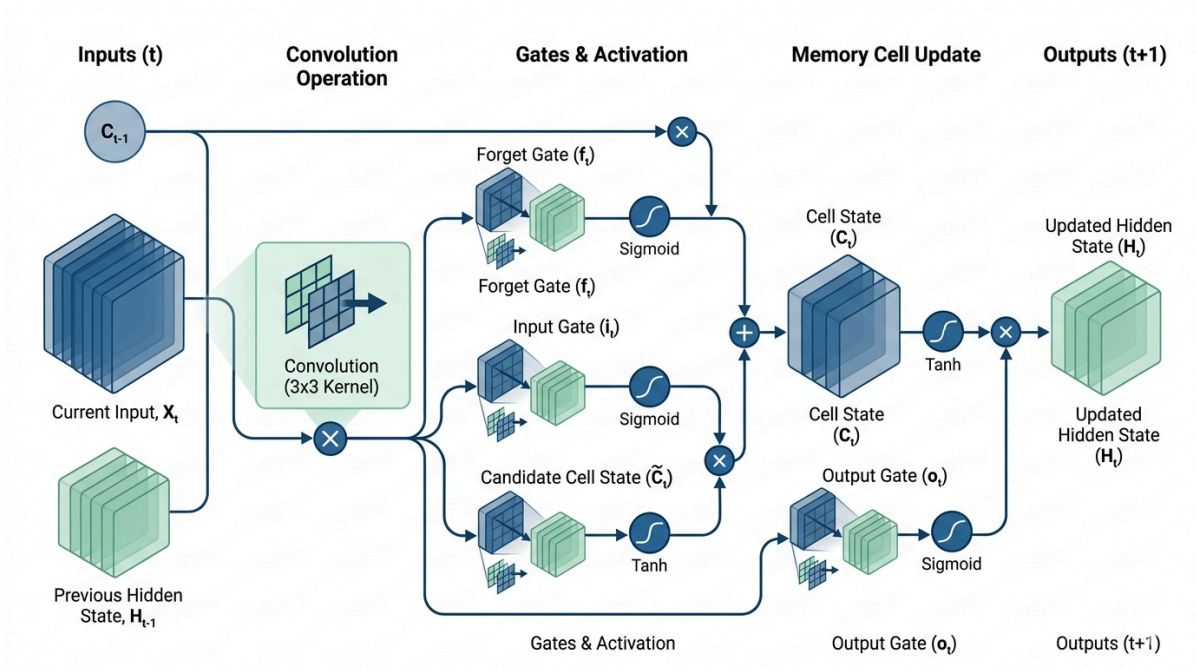
- Wx ：全连接、矩阵乘法
- x_t, h_t, c_t ：全部为一维向量

在改造后，ConLSTM的宏观结构和FC-LSTM（图二）一样，只是把所有的全连接改为了卷积：

- $i_t = \sigma(W_{xi} * x_t + W_{hi} * h_{t-1} + W_{ci} \circ c_{t-1} + b_i)$
- $f_t = \sigma(W_{xf} * x_t + W_{hf} * h_{t-1} + W_{cf} \circ c_{t-1} + b_f)$
- $\tilde{c}_t = \tanh(W_{xc} * x_t + W_{hc} * h_{t-1} + b_c)$
- $c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$
- $o_t = \sigma(W_{xo} * x_t + W_{ho} * h_{t-1} + W_{co} \circ c_t + b_o)$
- $h_t = o_t \circ \tanh(c_t)$

其中：

- $W * x$ ：卷积操作
- x_t, h_t, c_t ：都是二维特征图，保留了空间结构



图三⁴：ConLSTM的宏观结构

卷积起到的作用

在input-to-state中：

- 从输入到状态：当前时刻的输入 x_t 如何计算出当前时刻的状态 h_t/c_t ，从输入 x_t 到当前状态 h_t/c_t 的映射
- 卷积核在输入特征图上滑动，在局部感受野中进行计算，提取局部空间特征。
- 并且卷积的权重全局共享，其参数量远小于全连接。

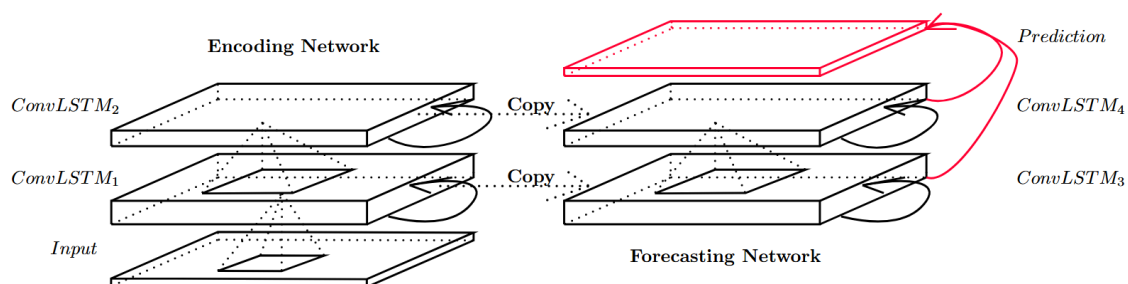
这使得模型在处理输入时候，可以保留图像的空间结构、提取空间特征

在state-to-state中：

- 上一时刻状态到当前时刻状态，进行时间递推
- 卷积核在上一时刻的cell上滑动，把历史的空间特征传递到当前时刻

使模型在传递时序记忆时，不会丢失空间结构，能对空间特征随时间的变化进行建模

具体应用架构



图四⁵：ConLSTM中的编码解码结构

整个模型由两部分组成，一是负责压缩历史时空特征的Encoder，另一个是负责进行预测生成未来帧的Forecaster\Decoder。

Encoder的隐藏层的输出信息直接复制到Forecaster的隐藏层，这使得上下文信息得以完整。

Forecaster的隐藏层会将 t 步的输出作为 $t+1$ 步的输入，喂给自己，继续进行预测。

后续问题和方向

- 对于近未来的预测（比如10分钟左右）很清晰，但是对较长时间后的预测（比如1小时后）边缘逐渐模糊。未来是多模态的，对于同一段历史序列，未来可能出现多种完全不同的发展路径。但是传统的 ConvLSTM 是确定性模型，它只会输出一个最妥当平均的结果，无法建模这些不同的可能性，只能用模糊的“平均状态”来掩盖不确定性。
- 卷积操作本质上是局部感受野的叠加。面对相隔较远的两个空间区域的相关性，纯粹的ConvLSTM需要很深的网络才能建立连接，效率依然有待提升。
- 后续可以考虑用注意力机制（比如Video Swin Transformer或者时空Attention）来替代ConvLSTM中的卷积操作，以解决其对全局空间依赖捕捉不足的问题。

附录

1. X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. Woo, "Convolutional LSTM network: A machine learning approach for precipitation nowcasting," in Advances in Neural Information Processing Systems (NIPS 2015), 2015, pp. 802–810. [↗](#)

2. AI工具生成 [↗](#)

3. 引用自: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/> [↗](#)

4. AI工具生成 [↗](#)

5. 引用自原论文中的Figure3 [↗](#)